Chap. 7
Requirements Analysis

Object Oriented Systems Analysis and Design Using UML, (4th Edition), McGraw Hill

Topics Covered

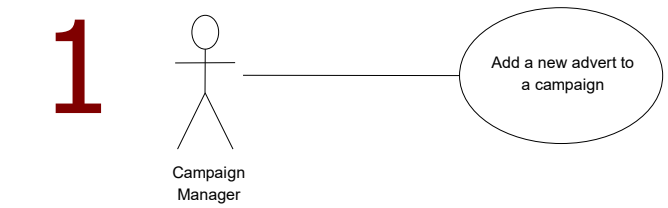
- Why do we analyze requirements ?
- Use Case Realization
- Class Diagrams
- Drawing a Class Diagram
- Assembling the Analysis Class Diagram

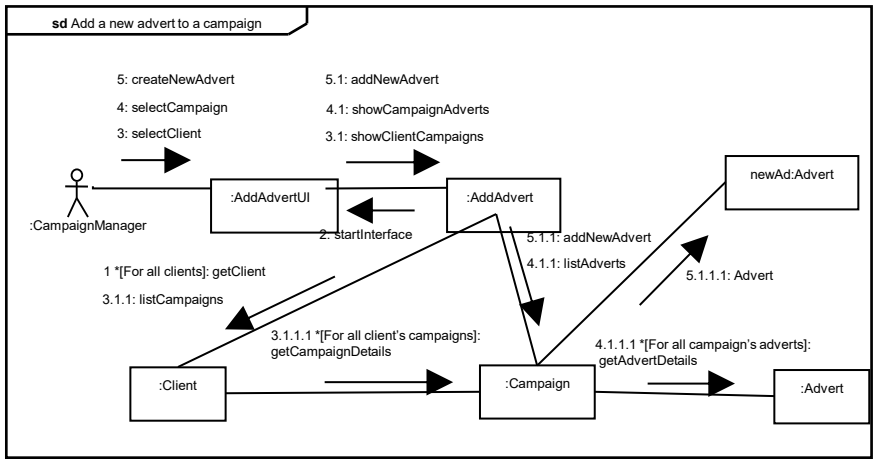
Why Analyse Requirements?

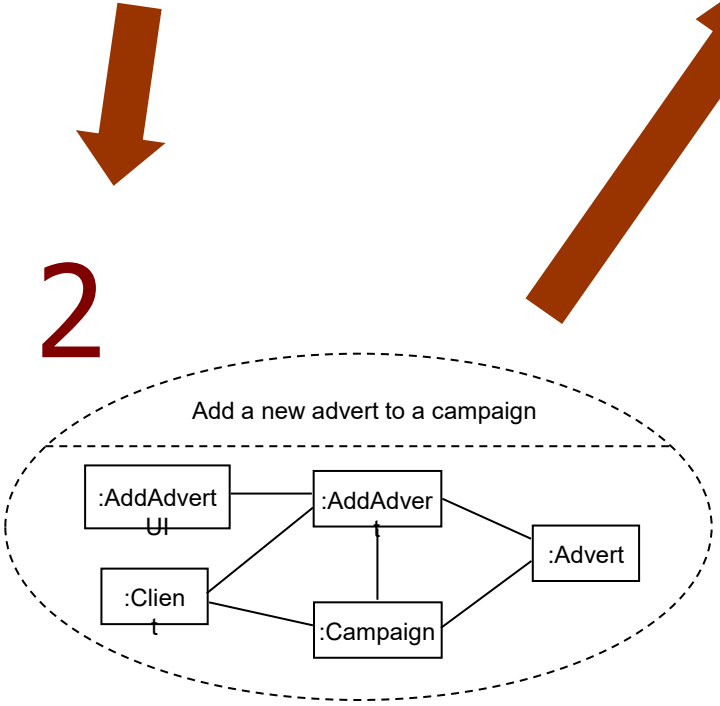
- Use case model alone is not enough
  - Move from use cases to analysis class diagrams
  - The analysis class diagram forms a basis for design class diagram
  - Use cases and class diagrams are linked by communication diagrams

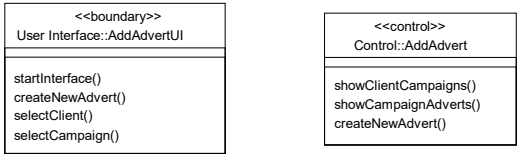
Use Case Realization

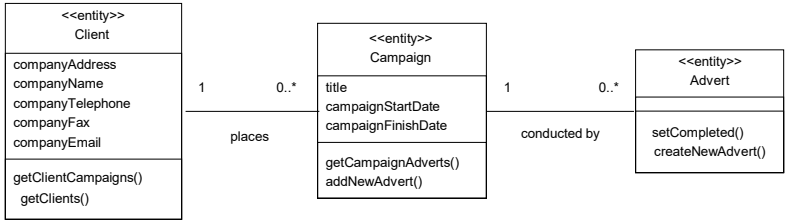
- To develop an abstract model into another model that is closer to its implementation





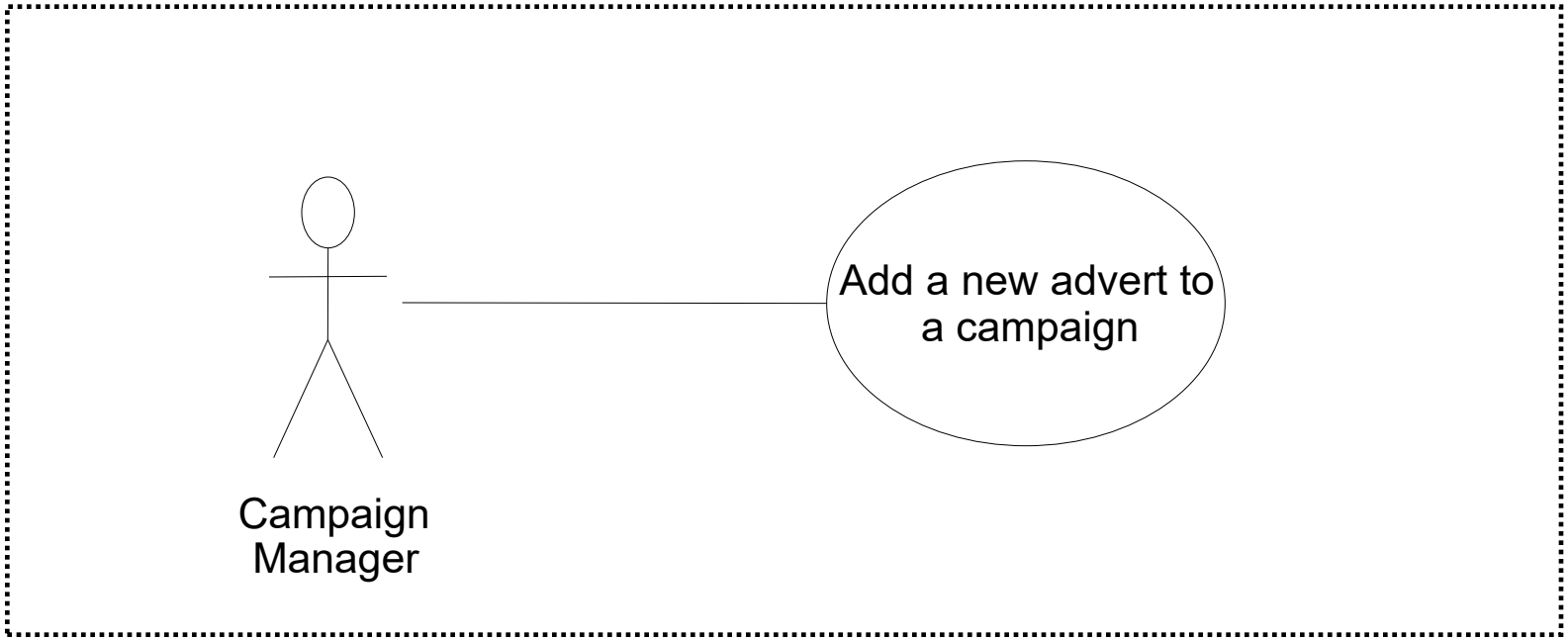






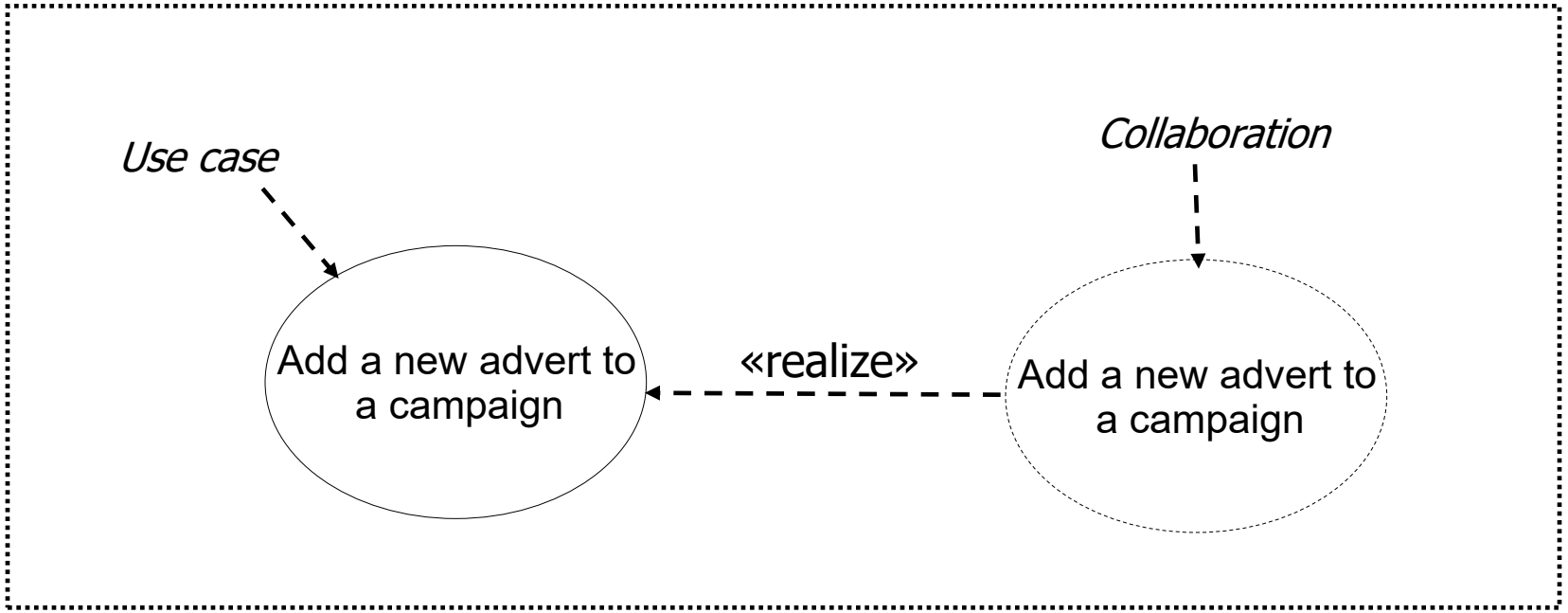
From use case to communication & class diagrams (1)

- Use case diagram for 'Add a new advert to a campaign'



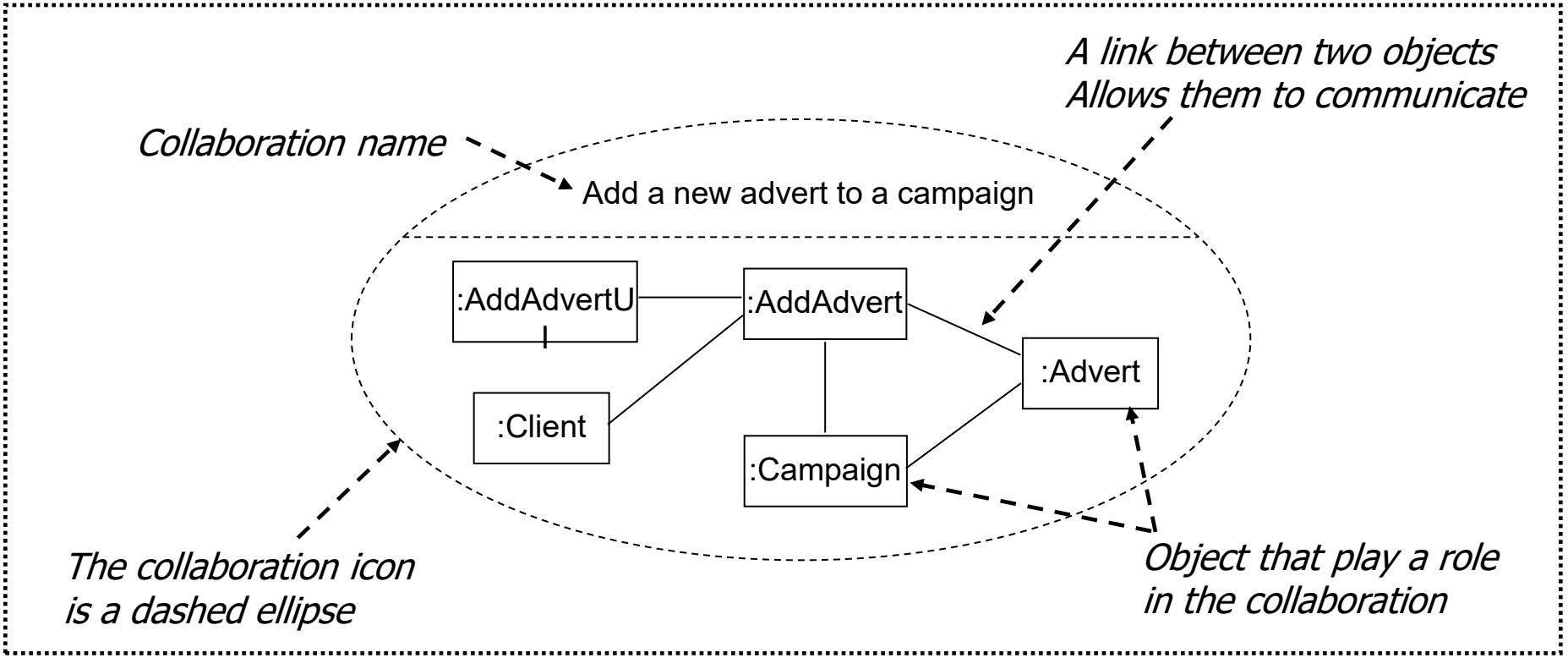
From use case to communication & class diagrams (2)

- A collaboration can realize a specific use case



From use case to communication & class diagrams (3)

- More detailed view of collaboration for 'Add a new advert to a campaign'



From use case to communication & class diagrams (4)

- Class stereotypes differentiate the roles objects can play:
  - Boundary objects
    - model interaction between the system and actors (and other systems)
  - Entity objects
    - represent information and behaviour in the application domain
  - Control objects
    - co-ordinate and control other objects

From use case to communication & class diagrams (5)

- Message labels in communication diagrams

Type of message	Syntax example
Simple message.	`4: addNewAdvert()`
Nested call with return value. *The return value is placed in the variable* `name`.	`3.1.2: name = getName()`
Conditional message. *This message is only sent if the condition* `[balance > 0]` *is true.*	`5 [balance > 0]: debit(amount)`
Iteration	`4.1 *[For all adverts]: getCost()`

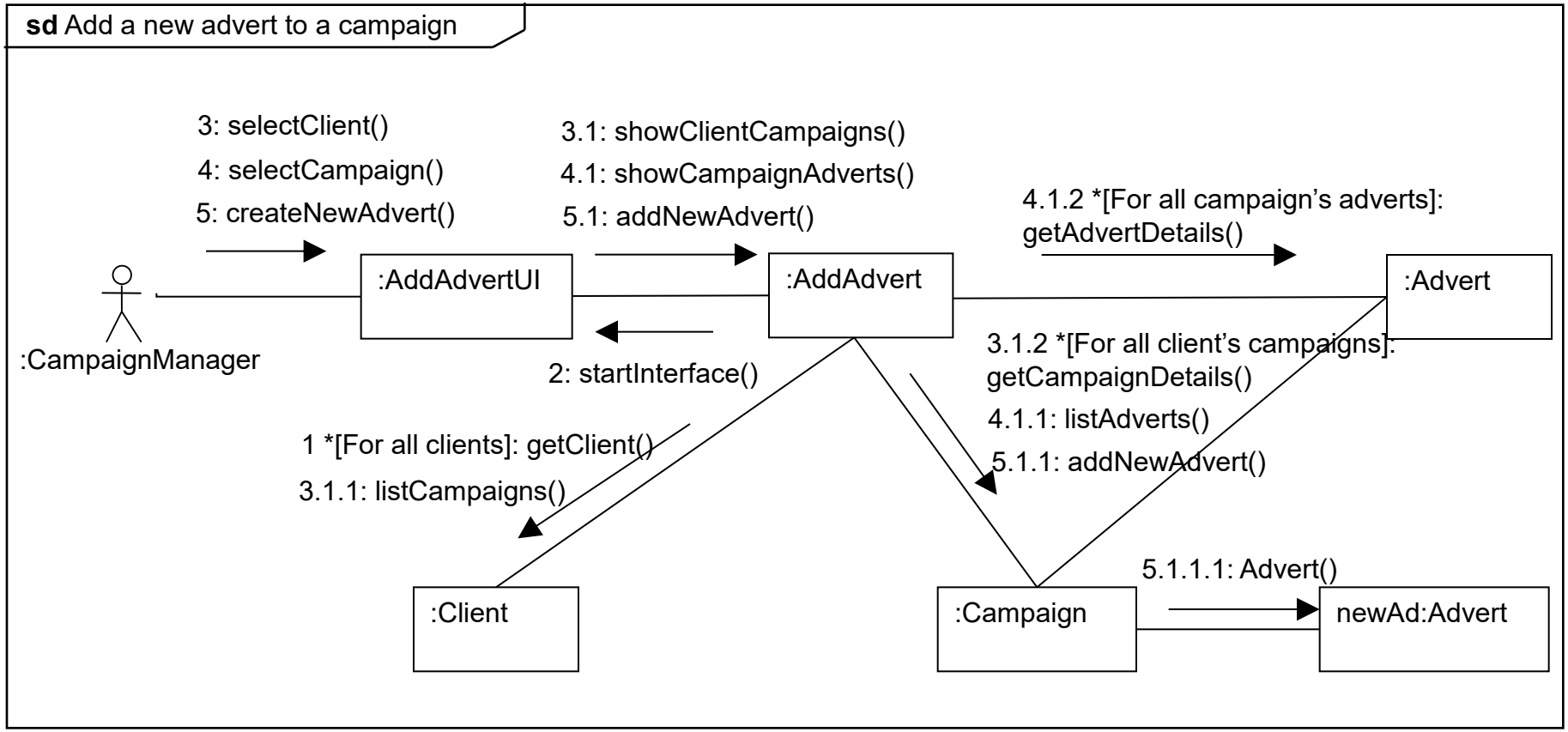
From use case to communication & class diagrams (6)

- Use case description
  - **Add a new advert to a campaign**

Actor action	*System response*
	1. Displays the list of all clients.
2. Selects the relevant client.	
	3. Displays the list of all campaigns belonging to the client
4. Selects the relevant campaign.	
	5. Displays the list of all adverts belonging to the campaign
6. Create a new advert after filling up necessary data fields.	
	7. Present a message confirming that the advert has been created.

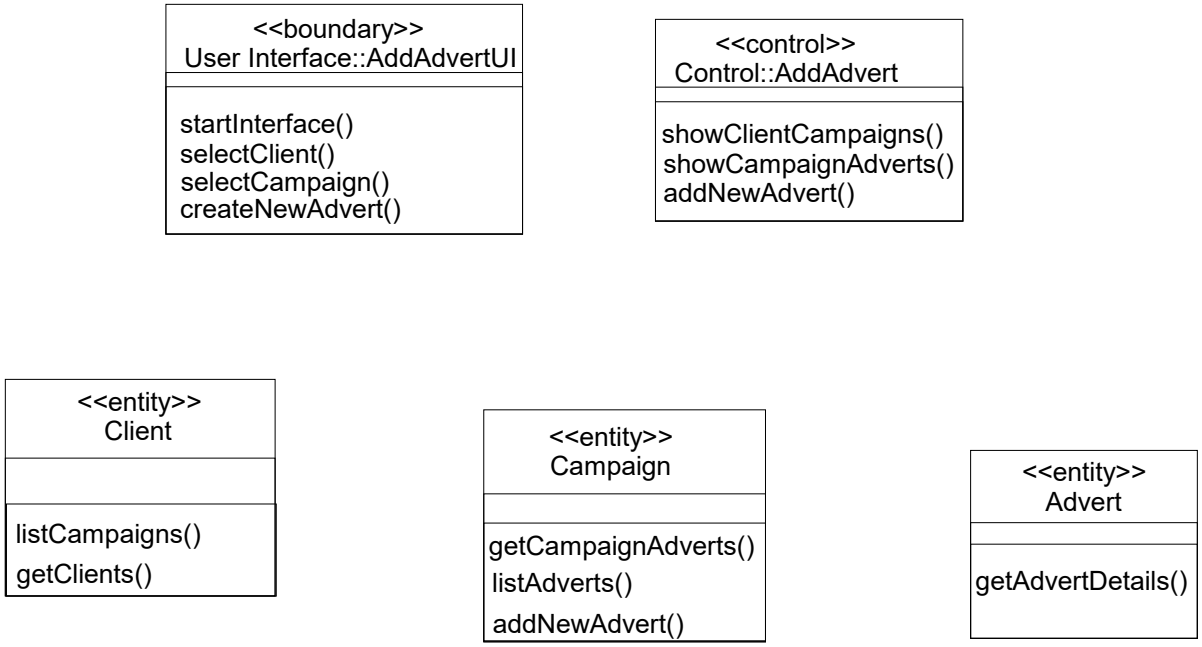
From use case to communication & class diagrams (7)

- Communication diagram for 'Add a new advert to a campaign'



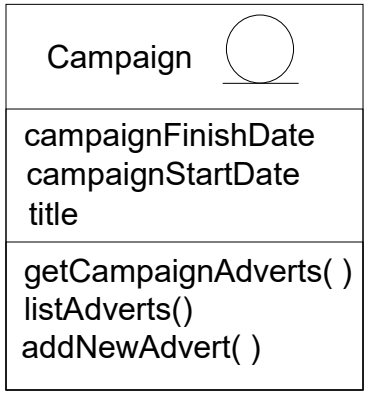
From use case to communication & class diagrams (8)

- Use case class diagram for 'Add a new advert to a campaign'

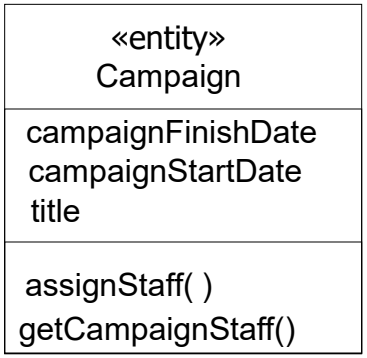


From use case to communication & class diagrams (9)

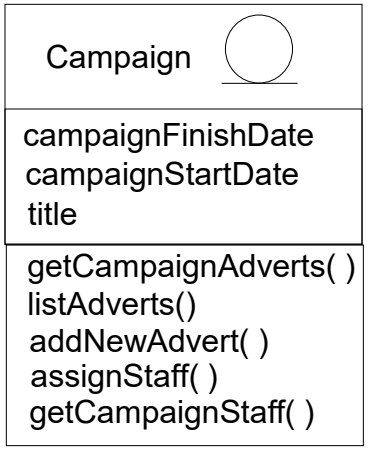
- Putting together different partial definitions of a class



(a) Meets the needs of
'Add new advert to a campaign'

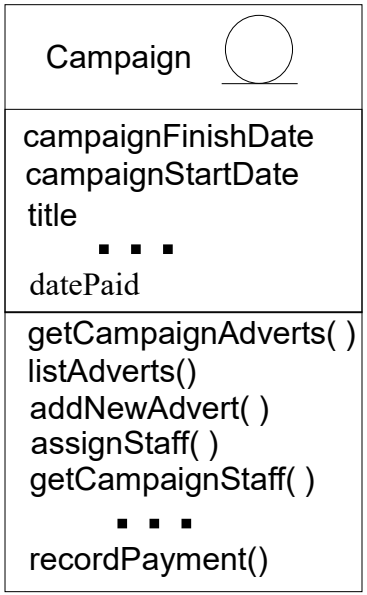


(b) Meets the needs of
'Assign staff to work on a campaign'



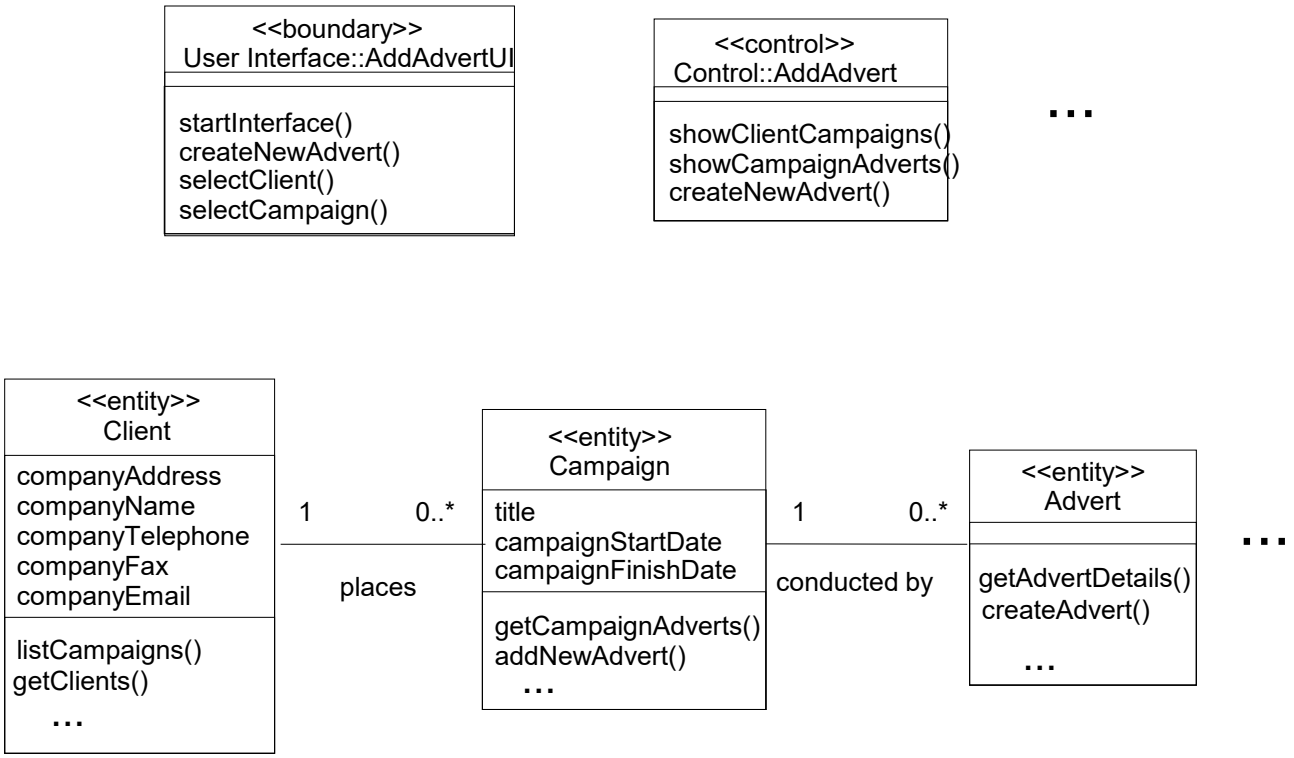
(c) Meets the needs of
both use cases

(d) A more fully developed class that meets these and other use cases

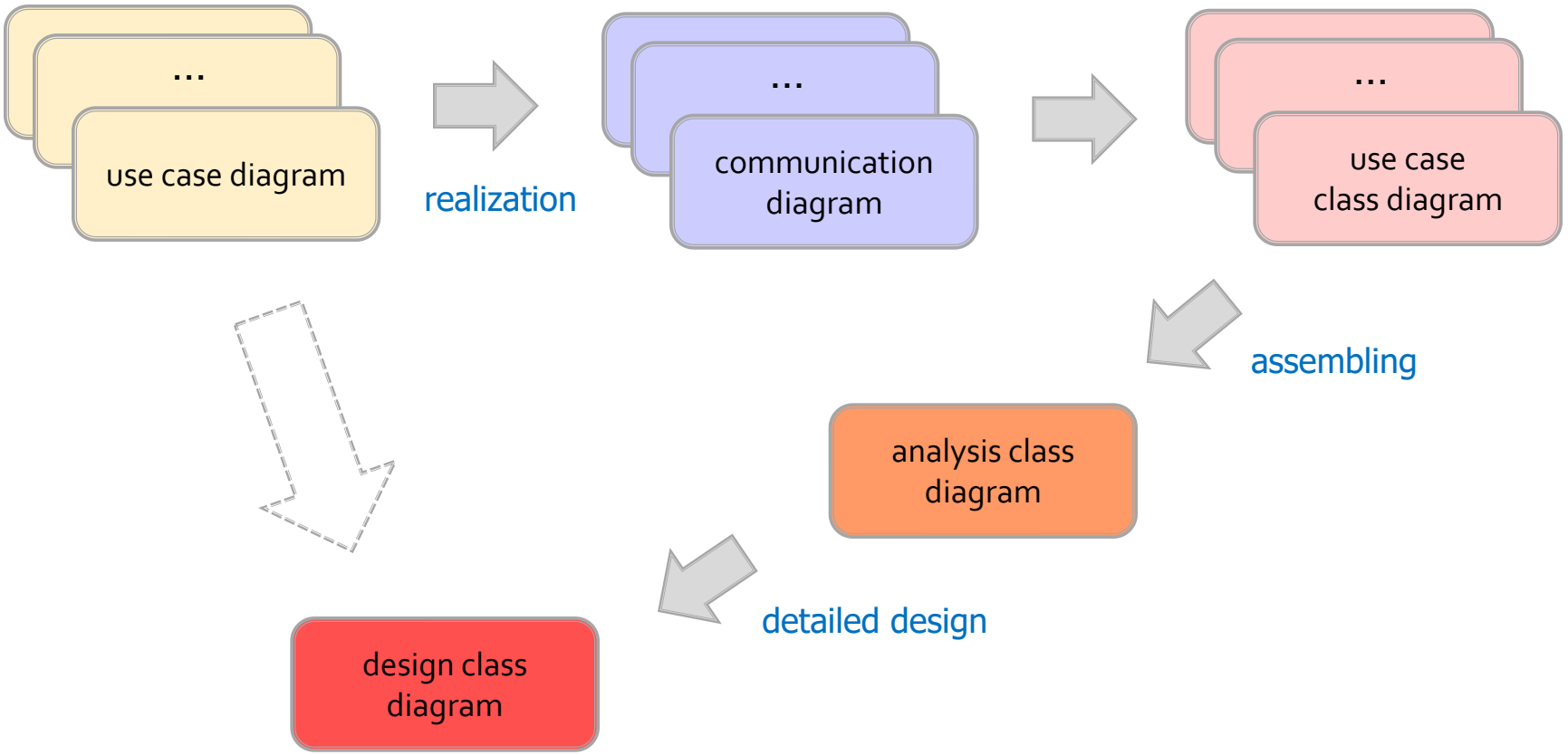


From use case to communication & class diagrams (10)

- Analysis class diagram for all classes

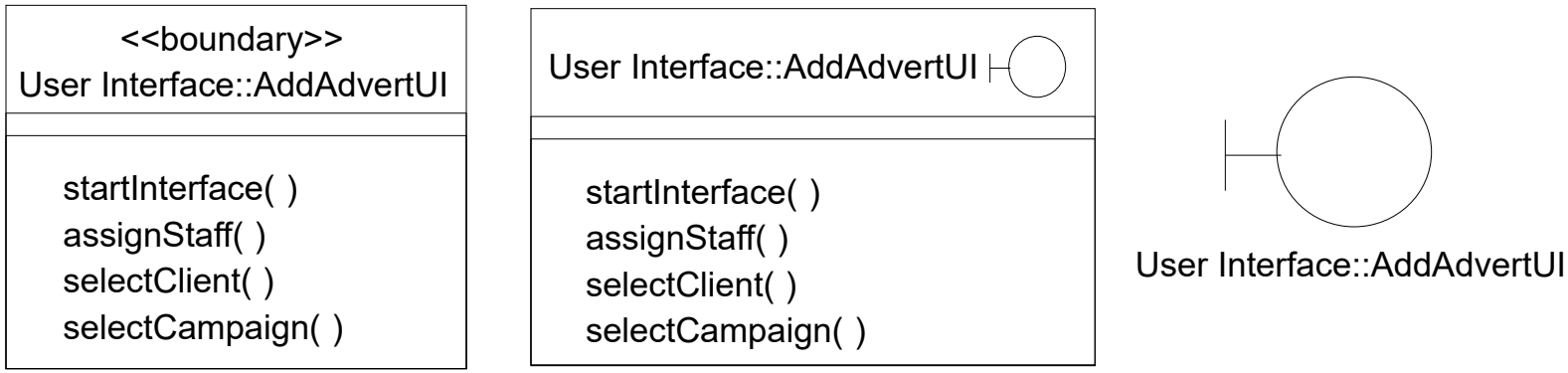


Designing flows with core diagrams



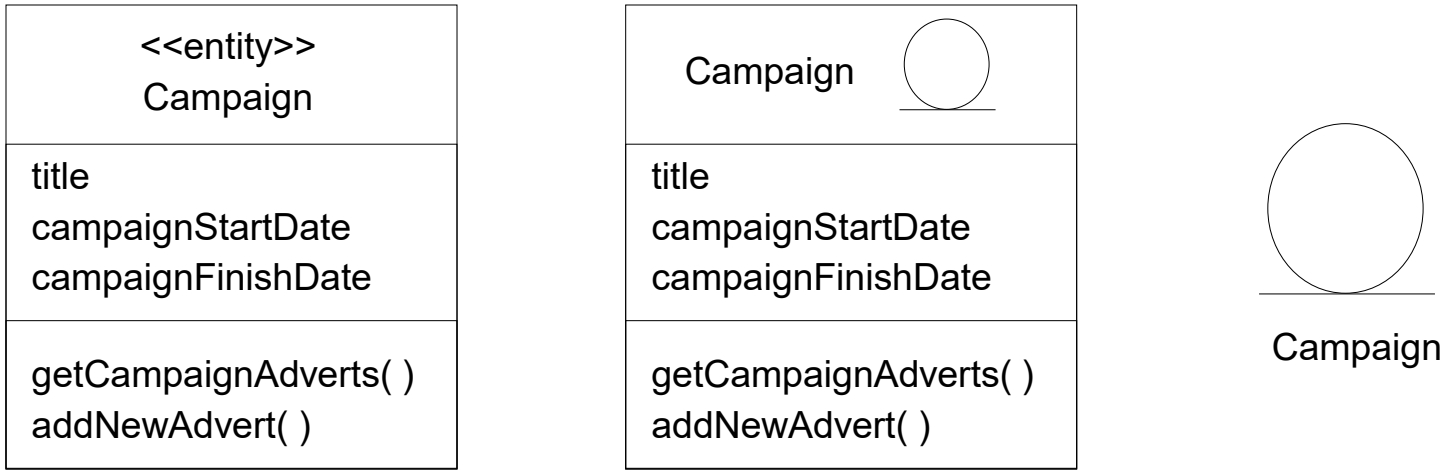
Class Diagram: Stereotypes (1)

Alternative notations for boundary class:



Class Diagram: Stereotypes (2)

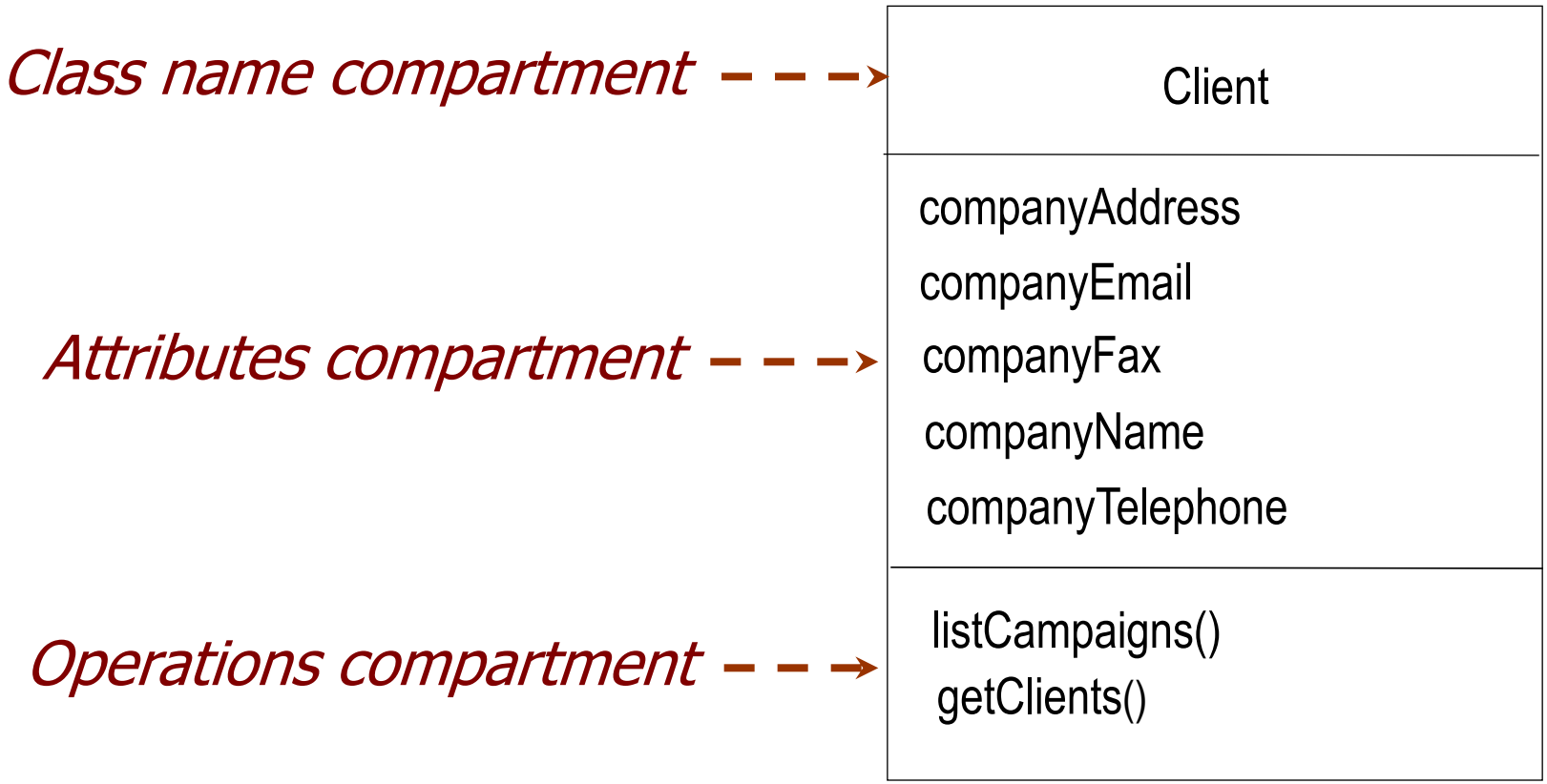
Alternative notations for entity class:



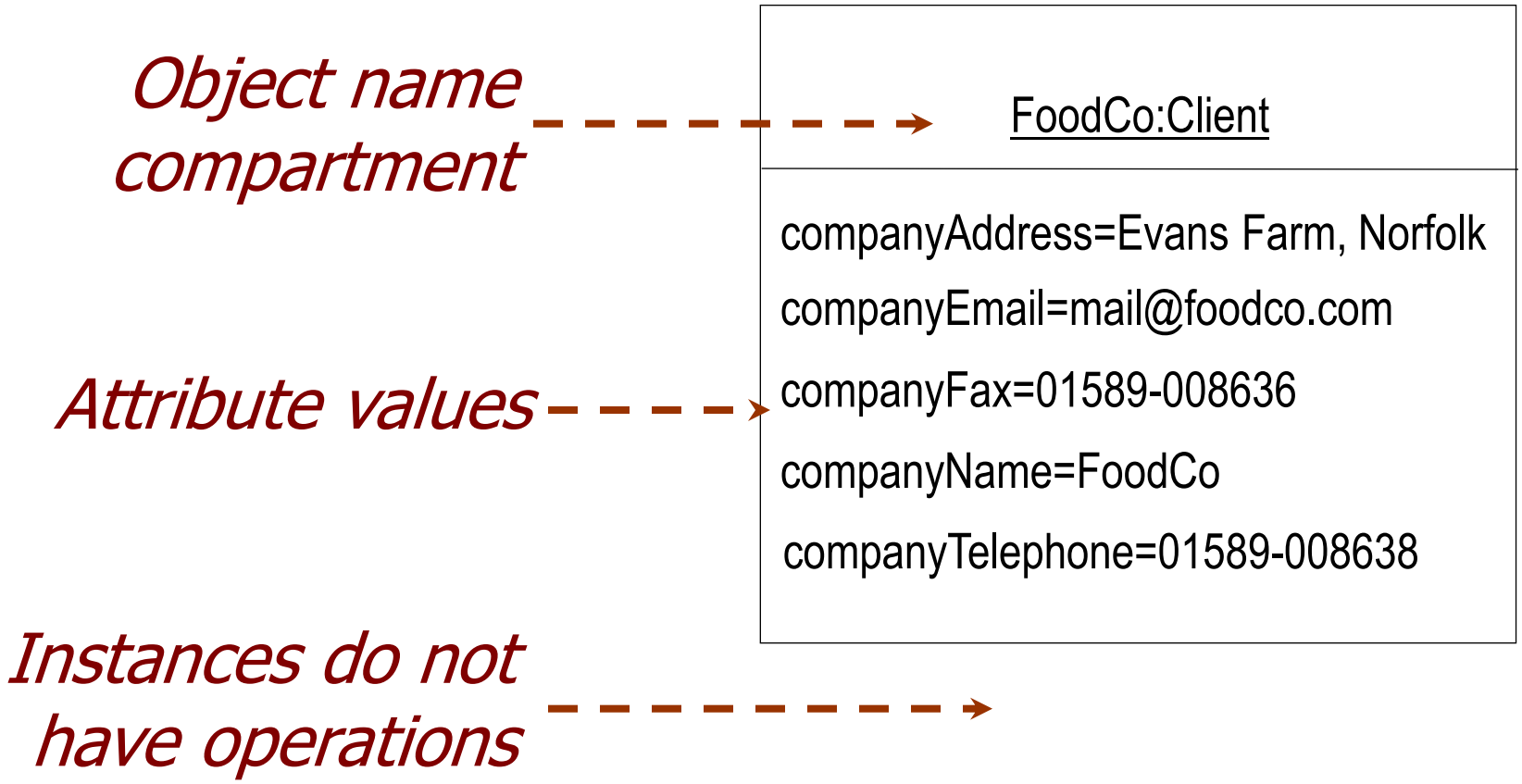
Class Diagram: Stereotypes (3)

Alternative notations for control class:

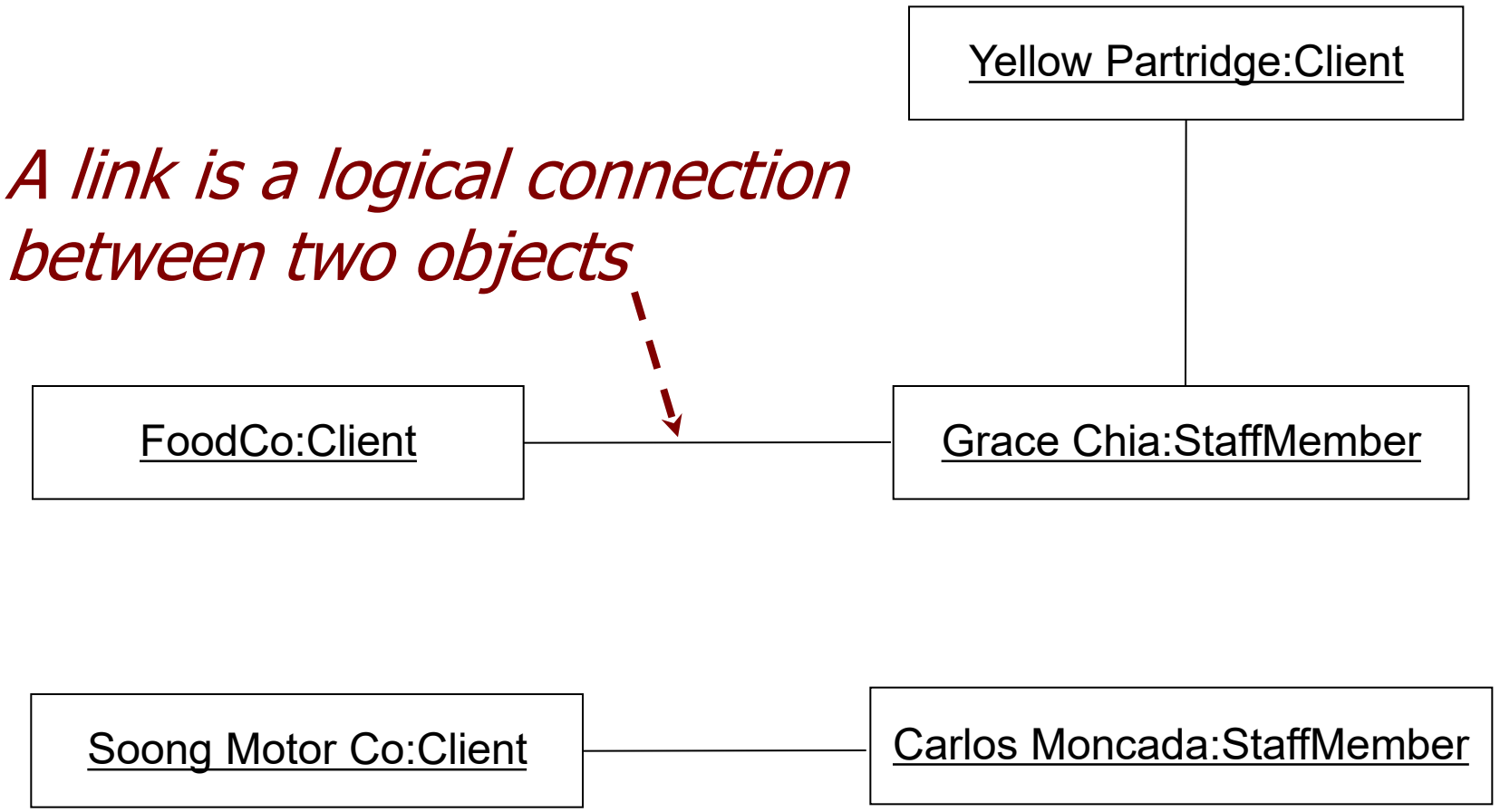
Class Diagram: Class Symbol



Class Diagram: Instance Symbol



Class Diagram: Links

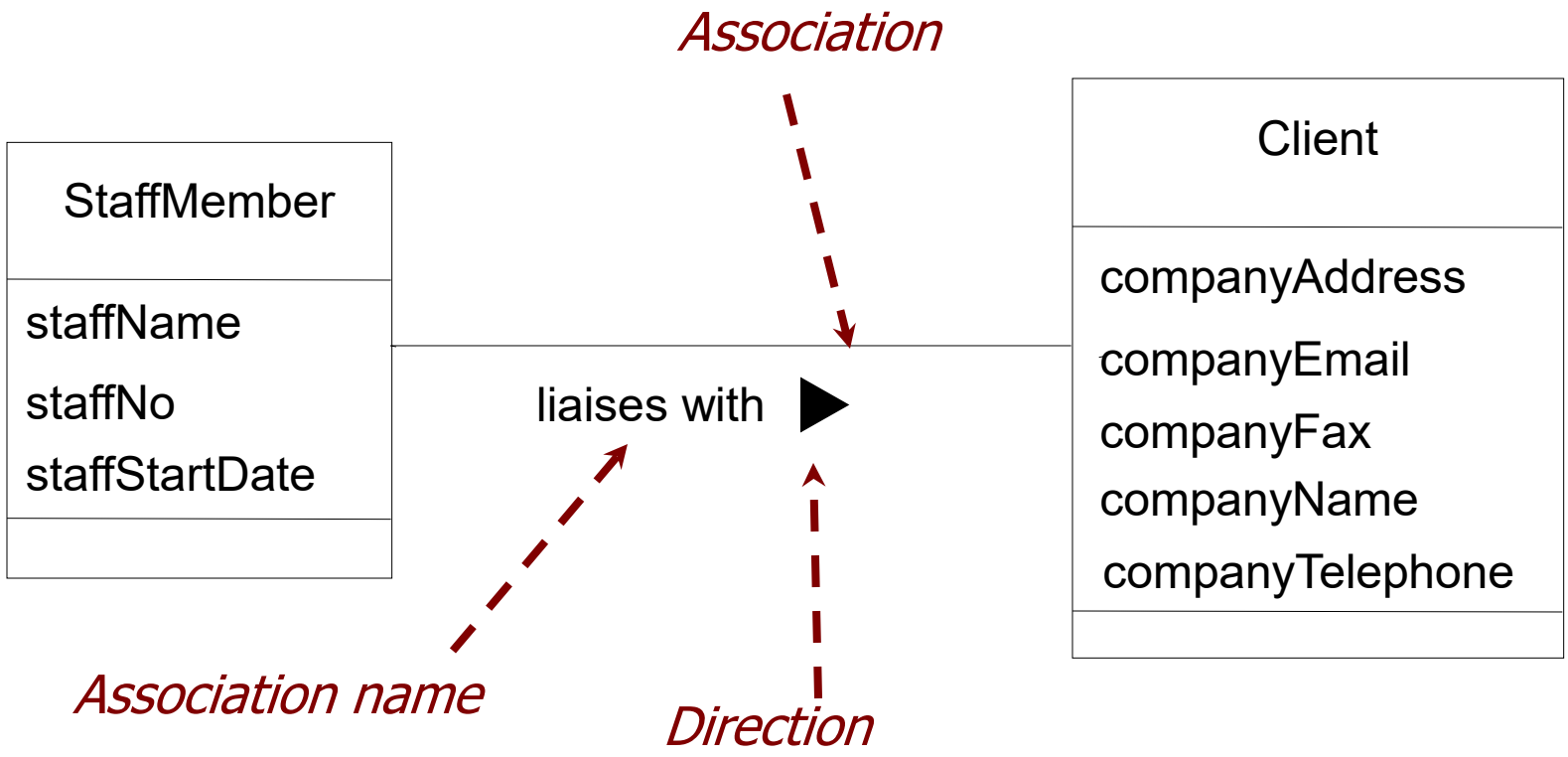


Class Diagram: Associations (1)

Associations represent

- The possibility of a logical relationship or connection between objects of one class and objects of another
- If two objects can be linked, their classes have an association

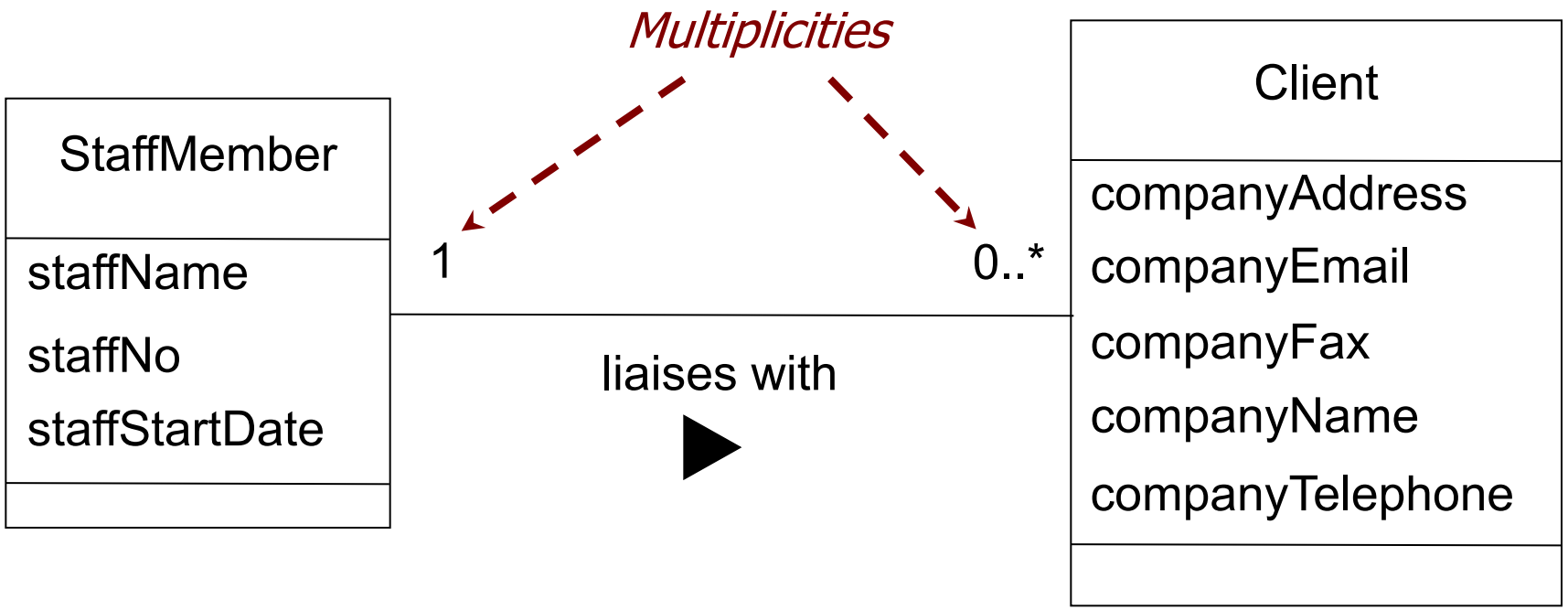
Class Diagram: Associations (2)



Class Diagram: Associations - Multiplicity (1)

- Associations have multiplicity
- Multiplicity is the range of permitted cardinalities of an association
- Represent *enterprise* (or *business*) *rules*
- For example:
  - Any bank customer may have one or more accounts
  - Every account is for one, and only one, customer

Class Diagram: Associations
- Multiplicity (2)



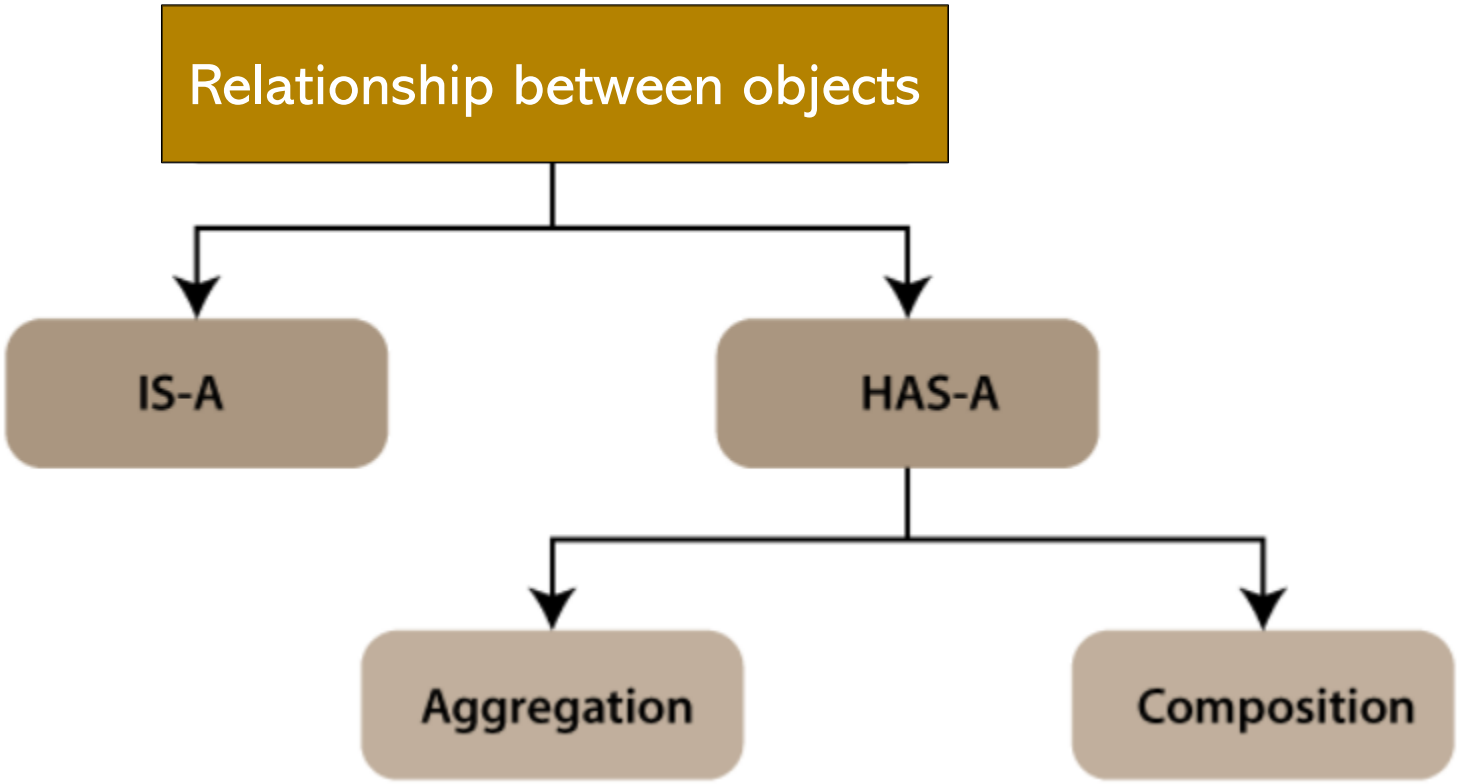
- Exactly one staff member liaises with each client
- A staff member may liaise with zero, one or more clients

Class Diagram: Associations
- Multiplicity (3)

Any range of values can be specified

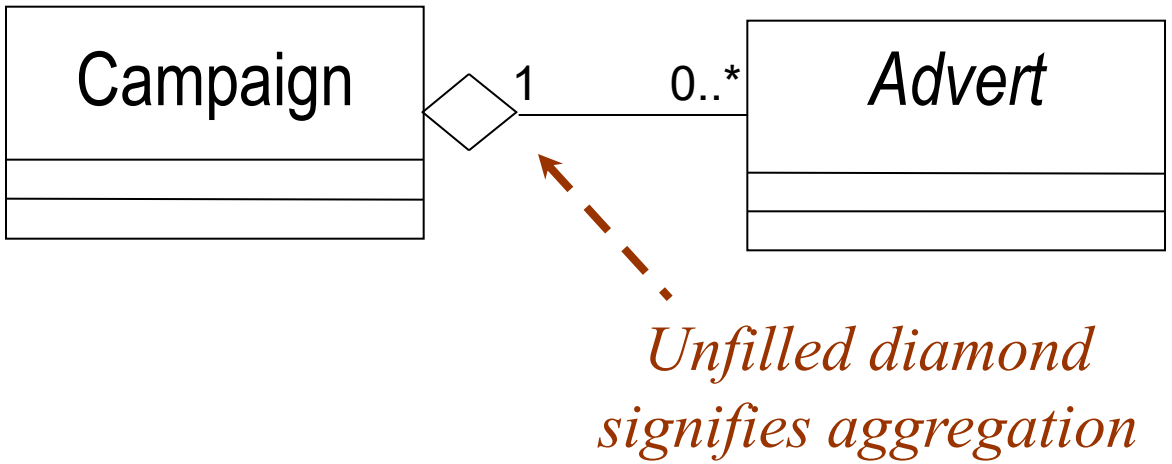
- 1..*
- 3..*
- 0..1
- 1..7
- 3, 5, 19
- 1, 3, 5..*

Class Diagram: Associations
- Relationship between objects



Class Diagram: Associations - Aggregation and Composition (1)

- Special types of association, both sometimes called whole-part
- A campaign is made up of adverts:



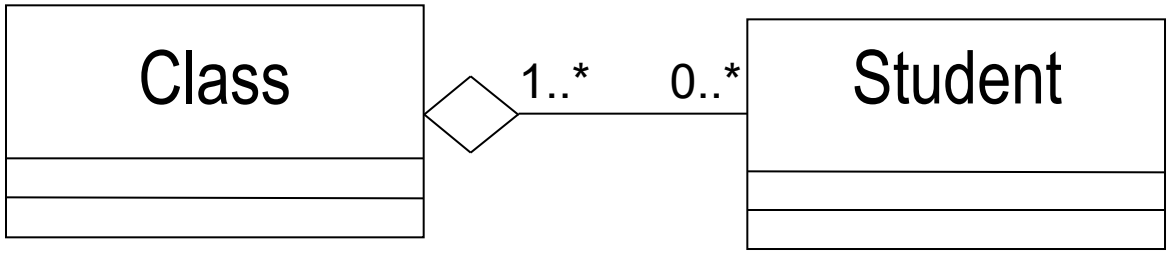
Class Diagram: Associations
- Aggregation and Composition (2)

- Aggregation is essentially any whole-part relationship

- Composition is 'stronger':
  - Each part may belong to only one whole at a time
  - When the whole is destroyed, so are all its parts

Class Diagram: Associations - Aggregation and Composition (3)

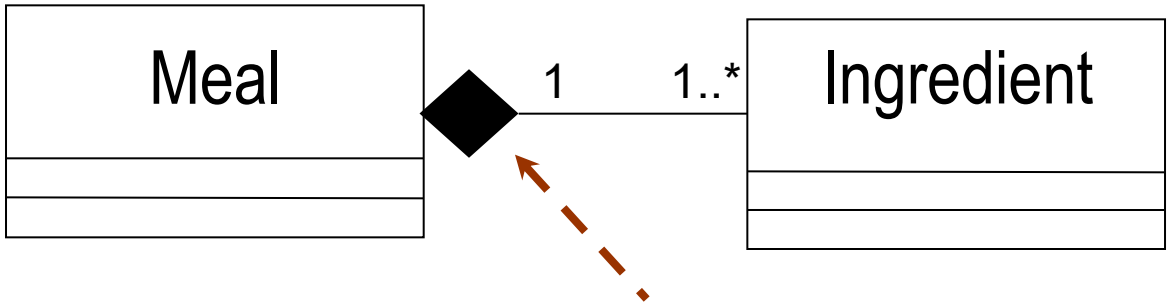
- An example:



  - Students could be in several classes
  - If class is cancelled, students are not destroyed!

Class Diagram: Associations - Aggregation and Composition (4)

- Another everyday example:



Filled diamond signifies composition

  - Ingredient is in only one meal at a time
  - If you drop your dinner on the floor, you probably lose the ingredients too

Class Diagram: Associations - Adding generalization Structure (1)

- Add generalization structures when:
  - Two classes are similar in most details, but differ in some respects
- Approach to finding generalization
  - Bottom-up approach
    - Look for similarities among classes
    - Then consider whether the model can be simplified by superclasses that abstract out the similarities
  - Top-down approach
    - Both superclasses and subclasses have already been identified
    - "a kind of" relationship

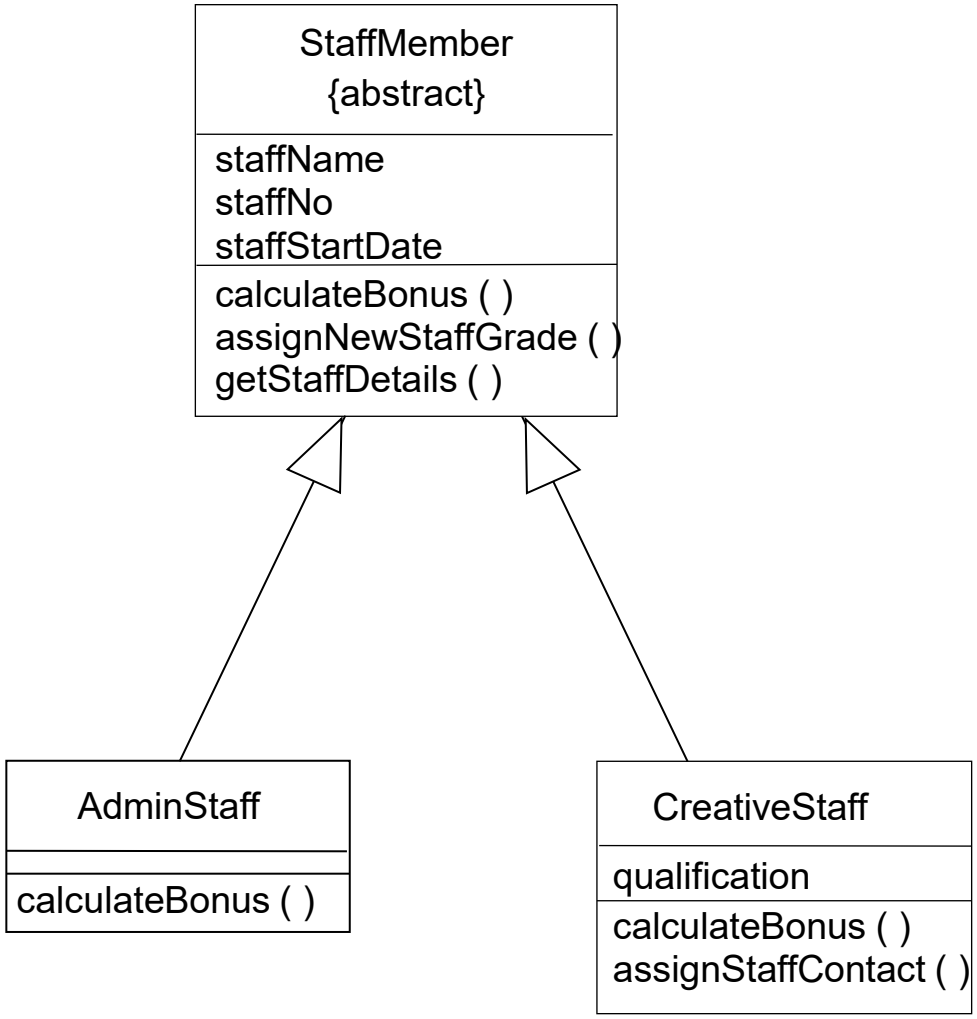
Class Diagram: Associations
- Adding generalization Structure (2)

- Two types of staff:

Creative	Have qualifications recorded Can be client contact for campaign Bonus based on campaigns they have worked on
Admin	Qualifications are not recorded Not associated with campaigns Bonus not based on campaign profits

Adding Structure:

- Operation is said to be redefined or overridden



Identifying classes

1. Start with one use case
2. Identify the objects involved in a use case collaboration
3. Draw a communication diagram that fulfils the needs of the use case
4. Translate this collaboration into a class diagram
5. Repeat for other use cases
6. Various use case diagrams are assembled into a single, combined analysis class diagram
   - Adding associations including aggregation, composition, and generalization